

TRAVAUX PRATIQUES III

Algorithme de Berry-Sethi

L'objectif de ce TP est d'implémenter l'algorithme de Berry-Sethi pour construire l'automate de Glushkov reconnaissant le langage dénoté par une expression régulière donnée. On apprendra également à manipuler des automates non déterministes, et notamment à les déterminer.

Ce TP utilise le langage OCaml.

A Préliminaires

A.1 Représentation

Dans tout le sujet, on utilisera le type suivant pour représenter une expression régulière :

OCaml

```

1 type 'a regex =
2   | Empty
3   | Eps
4   | Letter of 'a
5   | Union of 'a regex * 'a regex
6   | Concat of 'a regex * 'a regex
7   | Star of 'a regex

```

Remarque :

On pourra supposer que le type `'a` est totalement ordonné par la relation d'ordre usuelle de OCaml, et que les comparaisons entre éléments de type `'a` se font en temps constant. En pratique, `'a` sera égal soit à `char`, soit à `int` soit à `int * int`.

A.2 Outils fournis

Un fichier `nfa.ml` est fourni pour ce TP (à compléter). Il contient les types définis dans le TP et des fonctions utiles pour tester vos fonctions et vous amuser à visualiser vos résultats. **Il n'est pas nécessaire ni demandé de lire les fonctions déjà contenues dans ce fichier.**

Pour vous aider à tester plus facilement vos fonctions (et notamment vous permettre d'utiliser utop), tout est fourni et à écrire dans un même fichier `nfa.ml`.

Description des fonctions fournies :

- `parse : string -> int regex` prend en argument une chaîne de caractères représentant une expression régulière, par exemple `"(b|ca)*d"`, et renvoie l'expression régulière décrite, dans le type `int regex` ("a" devient 0, "b" devient 1, etc.). Notez qu'il y a deux caractères spéciaux : `#` représente $\emptyset$ et `&` représente ϵ .
- `string_of_regex : int regex -> string` fait l'opération inverse, elle renvoie une expression régulière sous forme de chaîne de caractères, facile à afficher.
- `graphviz_nfa : nfa -> string -> unit` prend en argument un `nfa` et un nom de fichier, et écrit dans le fichier la description graphviz de l'automate. Si le fichier n'existe pas, il est créé (cf précédent TP). Afin de visualiser facilement vos automates, il vous suffira donc de générer la description et de la copier-coller dans l'éditeur graphviz en ligne : <http://magjac.com/graphviz-visual-editor/>.

Exemple : Si `a` contient un `nfa`, lancer la commande `graphviz a "a.viz"` pour générer la description graphviz de `a` dans le fichier `a.viz`, puis copier-coller cette description dans l'éditeur en ligne.

Quelques exemples d'expressions régulières sont fournis à la fin du fichier `nfa.ml`.

1. Lisez la dernière partie du fichier `nfa.ml`, section (* ----- Exemples de regex à utiliser ----- *). Elle contient les exemples d'expressions régulières que vous pouvez utiliser librement pour vos tests, et un exemple d'utilisation de `string_of_regex`, que vous auriez tout aussi bien pu taper dans utop.

A.3 Fonctions utiles et premières manipulations

2. Écrire une fonction `merge : 'a list -> 'a list -> 'a list` qui prend en entrée deux listes u et v supposées strictement croissantes, et renvoie une liste strictement croissante dont l'ensemble des éléments est l'union de celui de u et de celui de v .
 $\triangle$ Attention au cas d'égalité, on ne veut pas de doublon dans cette liste.
3. Écrire une fonction `is_empty : 'a regex -> bool` qui teste si le langage dénoté par une expression est vide.
Indication : raisonnez par disjonction de cas (filtrage).
4. Écrire une fonction `contains_epsilon : 'a regex -> bool` qui teste si ϵ appartient au langage dénoté par une expression régulière.
5. Déterminer la complexité des trois fonctions précédentes.

B Langages locaux

6. Rappeler les définitions :
 - des ensembles P, S et F associés à un langage L ;
 - d'un langage local.
7. Écrire une fonction `prefix : 'a regex -> 'a list` qui prend en entrée une expression régulière e et renvoie l'ensemble P associé à $\mathcal{L}(e)$, sous forme d'une liste strictement croissante.
8. Écrire de même la fonction `suffix : 'a regex -> 'a list` calculant l'ensemble S .
9. Écrire une fonction `combine : 'a list -> 'b list -> ('a * 'b) list` qui prend en entrée deux listes `[x1; ...; xn]` et `[y1; ...; yp]` et renvoie une liste de longueur np contenant tous les couples (x_i, y_j) avec $1 \leq i \leq n$ et $1 \leq j \leq p$ (l'ordre des couples dans la liste est arbitraire).
10. Écrire la fonction `factor : 'a regex -> ('a * 'a) list` qui calcule l'ensemble F associé à une expression régulière.

C Automates de Glushkov

C.1 Linéarisation de l'expression

11. Écrire une fonction `number_of_letters : 'a regex -> int` qui prend en entrée une `'a regex` et renvoie le nombre total de feuilles de la forme `Letter x` qu'elle contient.
12. Écrire une fonction `linearize : 'a regex -> ('a * int) regex` qui prend en entrée un e de type `'a regex` et renvoie un e' de type `('a * int) regex` défini comme suit :
 - l'arbre e' a exactement la même forme que e ;
 - tous ses nœuds sont inchangés, à l'exception des feuilles `Letter x`;
 - chaque feuille `Letter x` est remplacée par une feuille `Letter (x, i)`, où i est son numéro (parmi les feuilles) dans l'ordre préfixe (i varie donc de 1 à `number_of_letters e`).

Exemple (dans utop) :

OCaml

```
1 let exemple =
2   Concat (Union (Letter 'c', Letter 'a'),
3           Star (Concat (Letter 'b',
4                       Union (Letter 'a', Letter 'c'))))
5
6 linearize exemple;;
7 - : (char * int) regex =
8 Concat (Union (Letter ('c', 1), Letter ('a', 2)),
9   Star (Concat (Letter ('b', 3), Union (Letter ('a', 4), Letter ('c', 5)))))
```

C.2 Construction de l'automate

13. Écrire une fonction `max_letter : int regex -> int` prenant en entrée une `int regex` et renvoyant le plus grand `i` tel que `Letter i` apparaisse dans l'expression, ou `-1` si l'expression ne contient aucune lettre.

Pour représenter un automate non déterministe, dans ce sujet on définit le type suivant :

OCaml

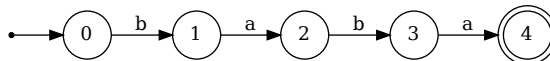
```
1 type state = int
2
3 type nfa =
4   {delta : state list array array;
5     accepting : bool array}
```

Cette représentation repose sur les propriétés suivantes, adaptées aux automates de Glushkov que l'on essaie ici de construire :

- Il y aura toujours un unique état initial, et ce sera toujours l'état `0`.
 - On se limite à des automates dont l'alphabet est de la forme $\llbracket 0 \dots m - 1 \rrbracket$ pour un certain m .
 - `delta` est une matrice de taille (n, m) , où n est le nombre d'états (et $m = |\Sigma|$). Pour un état q et une lettre x , `delta.(q).(x)` est la liste, sans doublon, codant l'ensemble $\delta(q, x)$.
 - `accepting` est un tableau de n booléens, et `accepting.(q)` indique si l'état q est acceptant.
 - L'automate est sans ϵ -transition.
14. Écrire une fonction `glushkov : int regex -> nfa` prenant en entrée une expression régulière e de type `int regex` et construisant l'automate de Glushkov associé à l'aide de l'algorithme de Berry-Sethi.

Indication : Si vous n'avez pas assez attentivement relu votre cours pour aujourd'hui, ressortez-le ! C'est la preuve de la proposition « Tout langage local est reconnaissable par un automate local standard » qui fournit la construction.

15. Générez la description graphviz de l'automate de Glushkov construit pour l'expression régulière `e4 = parse "baba"` et vérifiez que vous obtenez bien l'automate suivant :



Testez-le sur au moins l'une des deux expressions régulières `e2` ou `e3`.

D Automates non déterministes

D.1 Exécution d'un nfa

16. Écrire une fonction `delta_set` spécifiée comme suit :

Signature :

OCaml

```
1 val delta_set : nfa -> bool array -> int -> bool array
```

Arguments et préconditions :

- le premier argument est un automate non déterministe A d'ensemble d'états Q ;
- le deuxième argument est un tableau `arr` de n booléens, où $n = |Q|$, qui spécifie une partie E de Q ($q \in E$ si et seulement si `arr.(q) = true`);
- le troisième argument est un entier x correspondant à une lettre du langage de A (on garantit donc $0 \leq x < |a.delta.(0)|$).

Valeur de retour : un tableau de booléens codant la partie $\delta(E, x)$ de Q .

17. Écrire une fonction `has_accepting_state : nfa -> bool array -> bool` qui détermine si un ensemble d'états, codé par un tableau de $|Q|$ booléens, contient l'un au moins des états acceptants d'un automate A .

18. Écrire une fonction `nfa_accept : nfa -> int list -> bool` qui détermine si un mot, représenté par une liste d'entiers, appartient au langage d'un automate non déterministe donné.
Indication : passer par une fonction intermédiaire `delta_set_star` qui implémente $\delta^(E, u)$ pour un mot u . Vous pouvez utiliser un accumulateur.*
19. Déterminer la complexité de cette fonction. Quel avantage y aurait-il à utiliser une table de hachage au lieu d'un `bool array` pour représenter les ensembles d'états ?

D.2 Calcul de l'automate des parties

On définit un type pour les automates déterministes :

OCaml

```
1 type dfa =
2   {delta_d : state array array;
3   accepting_d : bool array}
```

- À nouveau, `delta_d` est une matrice dont les deux dimensions fournissent le nombre n d'états et le cardinal m de l'alphabet, qui est systématiquement $\llbracket 0 \dots m - 1 \rrbracket$.
- L'état initial est systématiquement l'état 0.
- `accepting_d` code les états acceptants, de manière similaire au champ `accepting` d'un `nfa`.
- Les champs ont été renommés `delta_d` et `accepting_d` pour éviter les ambiguïtés de typage.

Pour construire l'automate des parties, nous allons à nouveau être amenés à manipuler des ensembles d'états d'un automate non déterministe (un tel ensemble correspondant à un état de l'automate déterministe). Cette fois, nous les coderons par des listes strictement croissantes : l'ensemble $\{1, 4, 5\}$ sera codé par `[1; 4; 5]`, de manière à pouvoir les utiliser de manière sûre comme clés dans une table de hachage.

20. Expliquer pourquoi il est dangereux d'utiliser une structure mutable (comme un `array`) comme clé dans une table de hachage.
21. Écrire une fonction `build_set : nfa -> state list -> int -> state list` qui prend en entrée un `nfa`, un ensemble d'états s codé par une liste strictement croissante, et une lettre x , et renvoie l'ensemble d'états $\delta(s, x)$, toujours codé par une liste strictement croissante. On garantira une complexité en $O(|Q| + \deg_+(s, x))$, où $\deg_+(s)$ est le nombre de transitions de la forme $q \xrightarrow{x} q'$ avec $q \in s$.

Pour déterminer l'automate, nous allons procéder comme expliqué ci-dessous.

- 1) Dans un premier temps, l'objectif sera de construire deux structures de données.
 - Une table de hachage `sets`, dont les clés seront les états accessibles de l'automate des parties (sous forme de listes strictement croissantes d'états de l'automate non déterministe) et les valeurs les numéros de ces états dans l'automate déterministe. Ces numéros peuvent être arbitraires, tant qu'ils sont distincts et consécutifs à partir de zéro ; en pratique, ils correspondront à l'ordre dans lequel les états ont été explorés.
 - Une liste de triplets (i, x, j) correspondant aux transitions de l'automate déterministe. Un tel triplet indique la présence d'une transition $q_i \xrightarrow{x} q_j$, où i et j désignent les numéros des états (les valeurs dans la table de hachage, donc).
- 2) Dans un deuxième temps, il faudra construire effectivement l'automate déterministe, de type `dfa`, à partir des informations contenues dans les deux structures sus-citées, et éventuellement d'autres données auxiliaires calculées.

Pour les fonctions utiles du module `Hashtbl`, on se référera à la documentation.

22. Écrire la fonction `powerset : nfa -> dfa` qui prend en entrée un automate non déterministe et renvoie l'automate des parties associé. On ne construira que les états accessibles de cet automate, y compris l'ensemble vide s'il est accessible.

E Pour aller plus loin : Algorithme de McNaughton et Yamada

23. Rappeler le principe de l'algorithme de Floyd-Warshall, sa complexité et ses conditions d'application.
24. Retrouver, si possible sans se référer au cours, le principe de l'algorithme de McNaughton et Yamada pour le calcul d'une expression régulière dénotant le langage d'un automate. On rappelle que cet algorithme peut être vu comme une variante de celui de Floyd-Warshall.

25. Écrire une fonction `to_regex : nfa -> int regex` qui prend en entrée un automate non déterministe et renvoie une expression régulière dénotant son langage.